TMK Modeller Technical Documentation

AIC Team

The technical documentation for the TMK Modeller app.

May 05, 2026

Contents

1. Background: TMK Models 4
2. Application Overview 5
3. Features 6
   3.1. Three-Component Form Interface 6
   3.2. Live JSON Preview and Editing 6
   3.3. FSM Visualisation 6
   3.4. JSON Import and Export 6
   3.5. Day and Night Modes 6
   3.6. Comfortable Visual Design 6
4. Architecture & Internals 7
   4.1. Project Structure 7
   4.2. Technology Stack 7
      4.2.0.1. Common Components (`/common`) 8
      4.2.0.2. Forms (`/forms`) 8
      4.2.0.3. FSM Components (`/fsm`) 8
      4.2.0.4. Modal Components (`/modals`) 9
      4.2.0.5. Sections (`/sections`) 9
      4.2.0.6. Utilities (`/utils/helpers.ts`) 9
   4.3. Schemata 10
5. Branding & Visual Design 11
   5.1. Logotype 11
   5.2. Design Philosophy 11
   5.3. Theme Colours 11
6. Future Work 12
   6.1. Usability Improvements 12
   6.2. Native TMK Format Support 12
   6.3. High-Contrast / Accessibility Mode 12
   6.4. Validation and Linting 12

Bibliography 13

Index of Tables 14

1. Background: TMK Models

The Task–Method–Knowledge (TMK) modelling language is a formal knowledge representation framework originally developed for intelligent systems and AI agents, chiefly through research conducted at Georgia Tech's Design Intelligence Laboratory (DILab) under Prof. Ashok K. Goel and collaborators including Dr Spencer Rugaber, Dr Rahul K. Dass, and others on the Ivy and VERA teams.

TMK refers to the Task–Method–Knowledge language, where the 'Task' pertains to the cognitive goal of a skill, the 'Method' specifies the mechanism of completing the goal, and the 'Knowledge' refers to objects, concepts, and operations in the environment.

More precisely, TMK is a formal mechanism for describing a software system, created as a knowledge representation framework for intelligent systems. The main idea behind TMK is that effective reasoning about a system is supported by a description of both *what* a system does and *how* it works. TMK models are referred to as functional models because they focus on describing and decomposing the functions of systems.

A distinctive structural property is that TMK models are hierarchical: tasks can be decomposed into hierarchical sub-goals through their Method specification. In the Method specification, each state can have sub-goals, allowing a TMK structure to model an arbitrarily complex skill. Modelling skills in terms of states and transitions represents causality explicitly.

TMK has been applied prominently in the Ivy project — a generative AI coach that combines structured knowledge representations with large language models to deliver pedagogically aligned responses. Within that context, automating the construction of structured knowledge representations of procedural skills using the TMK framework, and leveraging generative AI to answer learners' How, Why, and What questions, is a central research concern.

Beyond pedagogy, TMK has a longer history in agent self-adaptation and self-explanation: the TMKL (Task–Method–Knowledge Language) supports modelling of an agent's composition and teleology. A TMKL model explicitly represents the tasks the agent addresses, the methods it applies, and the knowledge it uses. These models are used in a reasoning shell called REM (Reflective Evolutionary Mind), which enables the execution and incremental adaptation of agents that contain TMKL models of themselves.

A persistent practical obstacle has been the cost of authoring TMK models by hand. The manual creation of TMK models required approximately seven hours per model, including understanding the lesson (one to two hours), drafting the initial model (one to two hours), revising based on peer feedback (one hour), and finalising (one hour). A human-in-the-loop text-to-model pipeline cut down model drafting time, but expert refinement still required editing JSON models in text editors. The TMK Modeller described in this document is a direct response to that limitation.

---

2. Application Overview

The TMK Modeller is a browser-based tool built with React and TypeScript that provides a structured, form-driven interface for authoring and editing TMK model components. Its immediate motivation is the friction involved in writing or editing TMK models directly in a text editor or raw JSON file — a task that is error-prone and inaccessible to domain experts who are not necessarily programmers (and an unnecessary cognitive overhead for programmers as well).

Large language models can generate draft TMK models at speed, but those drafts invariably require expert review and correction before they are fit for use. The Modeller is designed to sit at that editing stage: it gives a subject-matter expert a comfortable, guided interface through which they can inspect, correct, and extend a draft model without needing to understand the underlying syntax. That said, even technically proficient users benefit from the interface; hand-editing nested JSON is tedious for anyone.

The application is structured around three tabs, one per TMK component (Task, Method, Knowledge). Each tab hosts a dedicated form and a live JSON preview that updates as the user fills in fields. A user may author a model entirely from scratch within the Modeller, or load (an) existing JSON(s) for editing. Completed components can be saved back to JSON for downstream consumption.

---

3. Features

3.1. Three-Component Form Interface

The application provides three independent tabbed forms — one each for the **Task**, **Method**, and **Knowledge** components of a TMK model. Each form is tailored to the schema of its respective component and exposes only the fields appropriate to that component, ensuring each component is 'correct-by-construction'.

3.2. Live JSON Preview and Editing

Each tab includes a real-time JSON preview pane (`MonacoPane.tsx`) that reflects the current state of the form. The preview updates as the user types, providing immediate feedback on how their inputs map to the underlying JSON structure. This is particularly useful for users who wish to cross-check the generated output against their expectations. In a major upgrade, the JSON preview pane is now a full Monaco editor element with day/night modes (akin to VS Code). Tech-savvy users can edit JSON directly; changes sync automagically with the form interface.

3.3. FSM Visualisation

Non-atomic support direct FSM visualisation. Clicking the eye icon opens **direct manipulation FSM graph editor**. Users may drag nodes, connect them with transitions (as in Unreal's Kismet/Blueprint), set start/success/failure states via RMB on states, and assign conditions by clicking over transitions. All changes sync back to the form and therefore the internal JSONs.

3.4. JSON Import and Export

Users may load a previously saved JSON file into any of the three forms. To guard against mistakes, the application validates that the loaded file corresponds to the correct tab: a Knowledge JSON cannot be inadvertently loaded into the Task form, and so forth. A warning is displayed if there is a mismatch. Completed forms can be saved to JSON at any point.

3.5. Day and Night Modes

The interface supports both a light (day) and a dark (night) mode. The primary theme colour shifts between the two modes (see §5 Branding for colour details). The toggle is accessible from the main application interface.

3.6. Comfortable Visual Design

The application deliberately avoids harsh contrasts in favour of comfortable, legible shades. There is currently no dedicated high-contrast mode (see §6 Future Work).

---

4. Architecture & Internals

4.1. Project Structure

```
/src
├── /components
│   ├── /common
│   │   ├── AssertionEntryFields.tsx   Reusable assertion input fields
│   │   ├── ConfirmDeleteButton.tsx    Button that confirms destructive actions
│   │   ├── DebouncedTextField.tsx     Text input with debounced onChange
│   │   ├── MonacoPane.tsx             Live JSON preview and editing
│   │   └── StringListField.tsx        Editable list-of-strings input
│   ├── /forms
│   │   ├── KnowledgeForm.tsx          Form for the Knowledge component
│   │   ├── MethodForm.tsx             Form for the Method component
│   │   └── TaskForm.tsx               Form for the Task component
│   ├── /fsm
│   │   ├── FSMEdge.tsx                React Flow edge for FSM transitions
│   │   ├── FSMNode.tsx                React Flow node for FSM states
│   │   ├── FSMToolbar.tsx             Toolbar above the graph canvas
│   │   ├── FSMView.tsx                Main FSM visualiser
│   │   └── fsm.utils.ts               Translates organiser object <--> React
Flow graph
│   ├── /modals
│   │   └── FSMModal.tsx               Hosts a method's organiser (visualisation)
│   ├── /sections
│   │   ├── KnowledgeEntries.tsx       Section-level Knowledge entry fields
│   │   ├── MethodEntryFields.tsx      Section-level Method entry fields
│   │   └── TaskEntryFields.tsx        Section-level Task entry fields
│   └── Visualiser.tsx                 Visualisation component
├── /utils
│   └── helpers.ts                     Shared utility functions
├── /schemata                          JSON schemata for Task, Method, Knowledge
├── App.css                            Global styles and theme variables
├── App.tsx                            Application root
├── App.test.tsx                       Top-level integration tests
├── index.tsx                          React DOM entry point
├── reportWebVitals.ts                 Performance reporting hook
└── setupTests.ts                      Test environment configuration
```

4.2. Technology Stack

Concern	Technology
UI framework	React 19
Language	TypeScript
Styling	CSS (App.css), component-scoped
Schema definition	JSON Schema (`/schemata`)
Web vitals	`reportWebVitals.ts`
Build tooling	Create React App

The choice of React with TypeScript provides strong type-safety across the component tree, which is important given the structured, schema-driven nature of the data being edited. TypeScript's static analysis catches category errors early.

4.2.0.1. Common Components (`/common`)

Component	Description
`AssertionEntryFields.tsx`	Input fields for assertional entries within a TMK component (e.g. preconditions, post-conditions). Reused across multiple form sections.
`DebouncedTextField.tsx`	A text input wrapper that delays the `onChange` callback, preventing excessive re-renders and live-preview updates on every keystroke.
`PreviewPane.tsx`	Displays the live JSON preview for the currently active form.
`StringListField.tsx`	A dynamic input for editing an ordered list of strings, with add/remove controls.

4.2.0.2. Forms (`/forms`)

Each form component corresponds to one TMK component and is responsible for rendering the complete set of fields required for that component's schema. Forms delegate to the `/sections` components for structured sub-sections and to `/common` components for atomic field types.

Component	TMK Component
`TaskForm.tsx`	Task
`MethodForm.tsx`	Method
`KnowledgeForm.tsx`	Knowledge

4.2.0.3. FSM Components (`/fsm`)

The FSM components implement the finite-state machine (FSM) visualisation and editing interface. They are built on React Flow and provide nodes, edges, and supporting utilities for interactive graph manipulation.

Component	Description
`FSMEdge.tsx`	Custom React Flow edge component representing FSM transitions.

Component	Description
`FSMNode.tsx`	Custom React Flow node component representing FSM states.
`FSMToolbar.tsx`	Toolbar displayed above the FSM canvas, offering controls such as zoom and layout reset.
`FSMView.tsx`	Main FSM visualiser; orchestrates rendering of nodes, edges, and user interactions.
`fsm.utils.ts`	Utility functions for translating between organiser objects and React Flow graph structures.

4.2.0.4. Modal Components (`/modals`)

The modal components currently consist of a single full-screen dialogue for FSM editing.

Component	Description
`FSMModal.tsx`	Near-full-screen modal that hosts `FSMView` for a specific Method's organiser. Provides a controlled open/close state, displays the method name, and propagates organiser edits upward via `onUpdate`.

4.2.0.5. Sections (`/sections`)

Section components handle logically grouped clusters of fields within a given form. They sit between the top-level form components and the atomic common components in the component hierarchy.

Component	Role
`TaskEntryFields.tsx`	Fields for individual Task entries (goal statements, preconditions, postconditions, etc.).
`MethodEntryFields.tsx`	Fields for Method entries, including FSM state/transition data.
`KnowledgeEntries.tsx`	Fields for Knowledge entries (objects, concepts, operations).

4.2.0.6. Utilities (`/utils/helpers.ts`)

Shared helper functions used across components — any logic that does not belong to a single component.

4.3. Schemata

The `/schemata` directory contains the JSON Schema definitions for all three TMK components. These schemata were a significant standardisation milestone in developing the Modeller: agreeing on the precise schema for each component determined the shape of every form, every validation rule, and the format of the files produced on export. They serve as the canonical source of truth for what constitutes a valid Task, Method, or Knowledge JSON.

The schemata are consumed at runtime to drive field rendering and import validation. They are not modified at runtime.

5. Branding & Visual Design

5.1. Logotype

5.2. Design Philosophy

The visual design favours comfort over austerity. Colours were chosen to be easy on the eyes across extended editing sessions, without resorting to stark, saturated primaries. The palette draws on natural imagery — mountain mist, bamboo, leaves.

There is currently no high-contrast mode; this is noted as a future accessibility consideration.

5.3. Theme Colours

The primary colour changes between day and night modes:

Mode	Hex	Name
Night (Dark)	`#3a895b`	山岚 (Shān Lán, ‘Mountain Mist’) سبزِ کوہسارِ مارگلہ (‘Margalla Mist Green’)
Day (Light)	`#115740`	竹漆 (Zhú Qī, ‘Bamboo Lacquer’) برگِ لائل پور (‘Lyallpur Leaf Green’)

Both shades sit within the green family: the night mode primary is a lighter, more luminous green suited to a dark background, while the day mode primary is a deep, rich green legible against light surfaces.

Theme variables are toggled at the application root level (`App.tsx`).

---

6. Future Work

The following capabilities are on the roadmap but are not present in the current version.

6.1. Usability Improvements

The *raison d'être* of the TMK Modeller is to offload the model author's cognitive load onto the interface. In keeping with this aim, there is room for usability improvements with the webform and graph editor interface to allow faster interactions without an increase in errors.

6.2. Native TMK Format Support

TMK models have a native textual representation (distinct from JSON) as defined by the TMKL specification. The Modeller currently reads and writes only JSON. Support for importing and exporting the native TMK/TMKL format is planned but not yet implemented.

6.3. High-Contrast / Accessibility Mode

The current design uses comfortable mid-range colours and does not include a high-contrast mode for users with visual impairments. An accessibility-focused theme is on the roadmap.

6.4. Validation and Linting

Beyond the current cross-tab import check, richer in-form validation (checking for semantic consistency between components, flagging missing required fields, warning about orphaned references between Task, Method, and Knowledge entries) would improve the authoring experience considerably.

This document describes the TMK Modeller as at the current release. For queries regarding the underlying TMK modelling framework, refer to the DILab–Ivy publications, including J. W. Murdock and A. K. Goel [1], A. K. Goel and S. Rugaber [2], C. Lum et al. *[3], R. K. Dass* et al. *[4], amongst others.*

Bibliography

[1] J. W. Murdock and A. K. Goel, "Meta-Case-Based Reasoning: Self-Improvement Through Self-Understanding," *Journal of Experimental & Theoretical Artificial Intelligence*, vol. 20, no. 1, pp. 1–36, 2008.

[2] A. K. Goel and S. Rugaber, "GAIA: A CAD-Like Environment for Designing Game-Playing Agents," *IEEE Intelligent Systems*, vol. 32, no. 3, pp. 60–67, 2017.

[3] C. Lum *et al.*, "Designing an AI Coaching System for Interactive Video-Based Skill Learning," in *Proceedings of the 21st International Conference on Intelligent Tutoring Systems*, Cham: Springer Nature Switzerland, 2025, pp. 281–291.

[4] R. K. Dass *et al.*, "Ivy: A Hybrid Knowledge-Based and Generative AI Coach for Explaining Procedural Skills," in *Artificial Intelligence in Education: 26th International Conference, AIED 2025, Palermo, Italy, July 22–26, 2025, Proceedings, Part II*, A. I. Cristea, E. Walker, Y. Lu, O. C. Santos, and S. Isotani, Eds., in Lecture Notes in Computer Science, vol. 15878. Springer, 2025, pp. 233–246. doi: 10.1007/978-3-031-98417-4_17°.

Index of Tables

Table 1 7
Table 2 8
Table 3 8
Table 4 8
Table 5 9
Table 6 9
Table 7 11